Laboratorio 2
Ejercicio 2: Citas Médicas

1. R# claros y verificables.

R#	Nombre	Descripción	Criterio de verificación
R1	mostrar menu principal	El sistema debe mostrar un menú con tres opciones: consultar doctores, pedir cita y salir, y repetirlo hasta que se elija salir.	Al ejecutar, se ven las 3 opciones. Tras completar una acción el menú reaparece. Una opción distinta de 1, 2 o 3 muestra un error y vuelve a pedir la opción.
R2	Registrar médicos del consultorio	El sistema debe tener registrados los 3 médicos (nombre, cédula, correo y título) al iniciar.	Se crean 3 instancias de Médico dentro del Consultorio con todos sus atributos asignados.
R3	Consultar doctores	El sistema debe mostrar en consola la información de los tres médicos tal como está registrada.	Al elegir la opción 1 se listan 3 médicos con nombre, cédula, correo y título, numerados del 1 al 3.
R4	Capturar datos del paciente	El sistema debe solicitar nombre, cédula, correo, teléfono y edad del paciente.	Se crea una instancia de Paciente con los 5 campos asignados.
R5	Validar datos del paciente	El sistema debe rechazar textos vacíos, edad fuera de [0, 120] y correos sin '@'.	Con una edad de -5 o 200, o un nombre vacío, se muestra el error y se pide el dato de nuevo hasta que sea válido.
R6	Solicitar fecha y hora	El sistema debe pedir la fecha (DD/MM/AAAA) y la hora (HH:MM) de la consulta y validar su formato	"31/02/2026" o "25:70" se rechazan y se vuelven a pedir. "15/11/2026" y "09:30" se aceptan.
R7	Seleccionar médico	El sistema debe permitir elegir uno de los 3 médicos por su posición (1 a 3).	Con 2 se selecciona a Camilo Ortaliz. Con 0 o 4 se rechaza y se pide de nuevo.
R8	Verificar disponibilidad	El sistema debe comprobar que el médico elegido no tenga otra cita en la misma fecha y hora.	Si ya hay una cita de ese médico en esa fecha y hora, se informa que no está disponible y no se registra.
R9	Registrar la cita	Si el médico está disponible, el sistema debe crear la Cita con paciente, fecha, hora y médico, y guardarla en la agenda.	Tras confirmar, la agenda contiene una cita más con todos sus campos.
R10	Confirmar y resumir la reserva	El sistema debe mostrar un mensaje de confirmación con el resumen (paciente, cédula, fecha, hora y médico).	Tras agendar se imprime "Cita confirmada" y los 5 datos del resumen.
R11	Salir	El sistema debe terminar la ejecución al elegir Salir.	Con la opción 3 el programa termina con código 0.

Requisitos NO Funcionales

R#	Nombre	Descripción	Criterio de verificación
RNF1	Estándar C++03	El código debe apegarse estrictamente a C++03.	Compila con `-std=c++03 -Wall -Wextra -Werror -pedantic`sin warnings ni errores.
RNF2	Guía de estilo	Sangría de 4 espacios, llaves Allman, nomenclatura formal y comentarios Doxygen.	Checklist visual: sin tabuladores y con llaves en su propio renglón.
RNF3	Modularidad y encapsulamiento	Interfaces (.hpp) separadas de implementaciones (.cpp) y atributos privados.	Existen medico.hpp/.cpp, paciente.hpp/.cpp, cita.hpp/.cpp, consultorio.hpp/.cpp, main.cpp y makefile.
RNF4	Legibilidad de la agenda	Los mensajes y listados deben ser claros para el usuario.	Cada médico y cada cita se imprimen en líneas etiquetadas (“Nombre:”, “Cédula:”, etc.).

2. Vocabulario

Término normalizado	Tipo	Definición	Sinonimos	¿Clase/ Atributo/ Operación candidata?
Consultorio	Entidad	Consultorio médico “Vivir Sano” que agrupa a los médicos y la agenda de citas.	Clínica, centro médico	Clase
Medico	Entidad	Profesional de la salud que atiende en el consultorio.	Doctor	Clase
Paciente	Entidad	Persona que solicita una cita.	Usuario, cliente	Clase
Cita	Entidad	Reserva de una consulta entre un paciente y un médico en una fecha y hora.	Reserva, consulta	Clase
nombre_consultorio	string	Nombre comercial (“Vivir Sano”).	Nombre	Atributo
TOTAL_MEDICOS	int	Número fijo de médicos (3) .	Cantidad de doctores	Atributo
nombre_medico	string	Nombre del médico.	Doctor	Atributo
cedula_medico	string	Cédula del médico.	Documento, ID	Atributo
correo_medico	string	Correo electrónico del médico.	Email	Atributo
titulo_medico	string	Título profesional y universidad.	Formación	Atributo
nombre_paciente	string	Nombre del paciente.	Nombre	Atributo
cedula_paciente	string	Número de cédula del paciente.	Documento	Atributo
correo_paciente	string	Correo del paciente.	Email	Atributo
telefono_paciente	string	Teléfono de contacto.	Celular	Atributo
edad_paciente	int	Edad en años cumplidos.	Años	Atributo
fecha	string	Dia de la consulta (DD/MM/AAAA).	Día	Atributo
hora	string	Hora de la consulta (HH:MM).	Horario	Atributo
citas_	vector<Cita>	Agenda con las citas registradas.	Agenda	Atributo
ConsultarDoctores	Operación	Lista de los 3 médicos del consultorio.	Ver médicos	Operación
EstaDisponible	Operación	Indica si el médico no tiene cita en esa fecha y hora.	Verificar disponibilidad	Operación
AgendarCita	Operación	Registra la cita si el médico está disponible.	Pedir cita	Operación
ObtenerMedico	Operación	Devuelve el médico de una posición dada.	Buscar médico	Operación

3. CRC

Campo	Contenido a completar
Clase	Médico
Responsabilidades (qué hace)	1. Almacenar nombre, cédula, correo y título. 2. Proveer getters de esos 4 datos. 3. Generar una descripción en texto de sí mismo.
Colaboradores (a quién le pide)	Ninguno
Mensajes clave (remitente → receptor.método(parámetros) → efecto/valor)	*medico.GetNombre() -> const std::string&,* *medico.ToString() -> std::string*

Campo	Contenido a completar
Clase	Paciente
Responsabilidades (qué hace)	1. Almacenar nombre, cédula, correo, teléfono y edad. 2. Proveer getters de esos datos.
Colaboradores (a quién le pide)	Ninguno
Mensajes clave (remitente → receptor.método(parámetros) → efecto/valor)	*paciente.GetNombre() -> const std::string&,* *paciente.GetCedula() -> const std::string&*

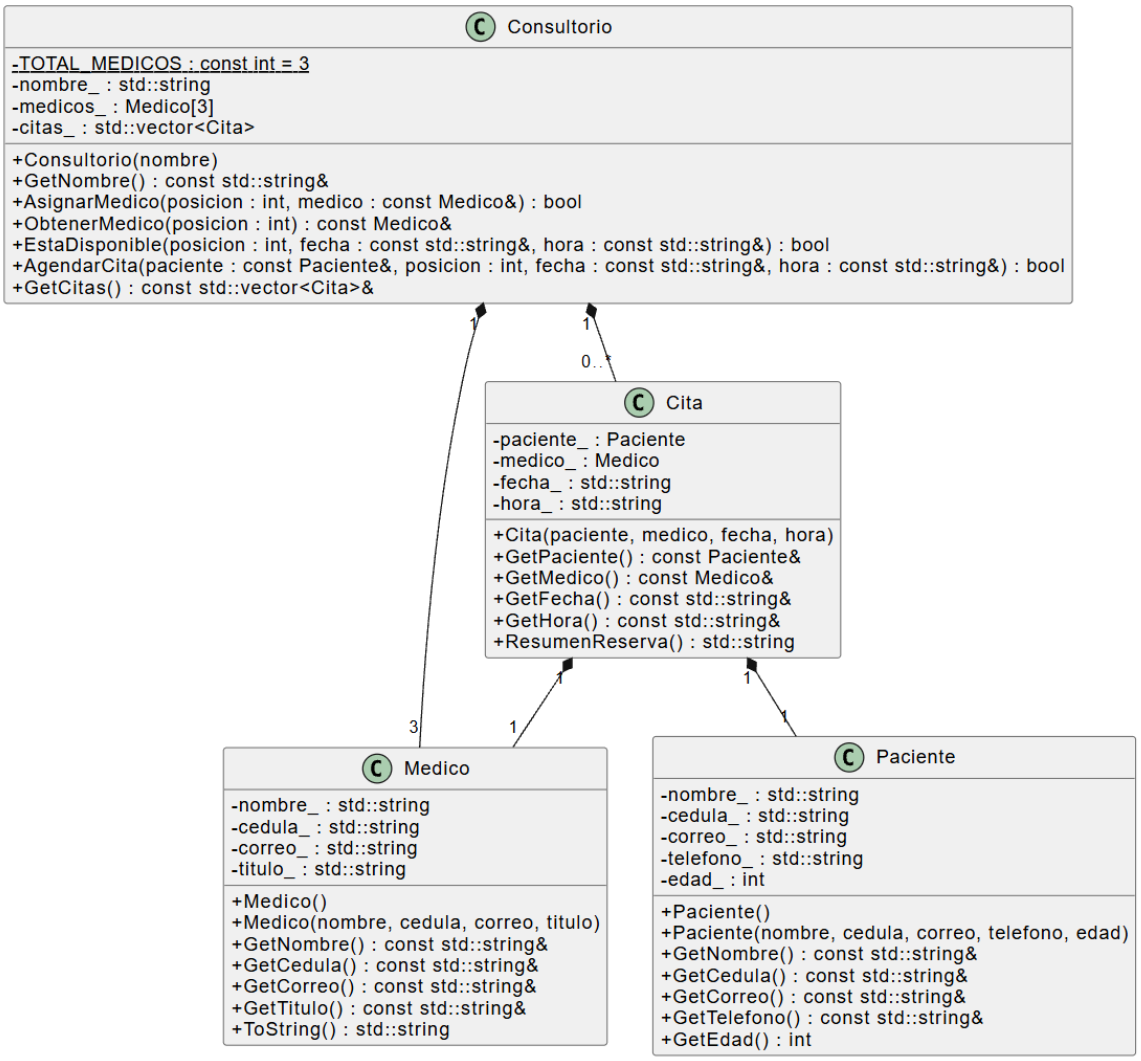
Campo	Contenido a completar
Clase	Cita
Responsabilidades (qué hace)	1. Registrar y conservar paciente, médico, fecha y hora de la consulta. 2. Facilitar métodos de lectura (Getters) para paciente, médico, fecha y hora. 3. Construir la representación textual estructurada con los datos de la reserva.
Colaboradores (a quién le pide)	Paciente, Medico (los contiene).
Mensajes clave (remitente → receptor.método(parámetros) → efecto/valor)	*`cita.GetFecha()`*, *`cita.GetHora()`*, *`cita.GetMedico() -> const Medico&`*, *`cita.ResumenReserva() -> std::string`*

Campo	Contenido a completar
Clase	Consultorio
Responsabilidades (qué hace)	1. Almacenar el nombre, un arreglo fijo de 3 médicos y la agenda de citas. 2. Consultar médicos por posición. 3. Verificar la disponibilidad de un médico. 4. Agendar la cita si está disponible.
Colaboradores (a quién le pide)	Medico, Cita, Paciente.
Mensajes clave (remitente → receptor.método(parámetros) → efecto/valor)	*ObtenerMedico(posicion: int) -> const Medico&,* *EstaDisponible(posicion: int, fecha: const string&, hora: const string&) -> bool,* *AgendarCita(paciente: const Paciente&, posicion: int, fecha: const string&, hora: const string&) -> bool, GetCitas() -> const vector<Cita>&*

Campo	Contenido a completar
Clase	Controlador(main)
Responsabilidades (qué hace)	1. Mostrar el menú y leer la opción (R1). 2. Capturar y validar los datos con reintentos (R4–R7). 3. Invocar al Consultorio para agendar (R8–R9). 4. Imprimir los médicos y la confirmación (R3, R10).
Colaboradores (a quién le pide)	Consultorio, Paciente, Medico, Cita.
Mensajes clave (remitente → receptor.método(parámetros) → efecto/valor)	*consultorio.ObtenerMedico(i),* *consultorio.AgendarCita(paciente, pos, fecha, hora)*

4. Diagrama de clase (UML)

Hecho en PlantUML (*https://www.planttext.com/*)



5. EPS/Contratos y 6. Pseudocódigo

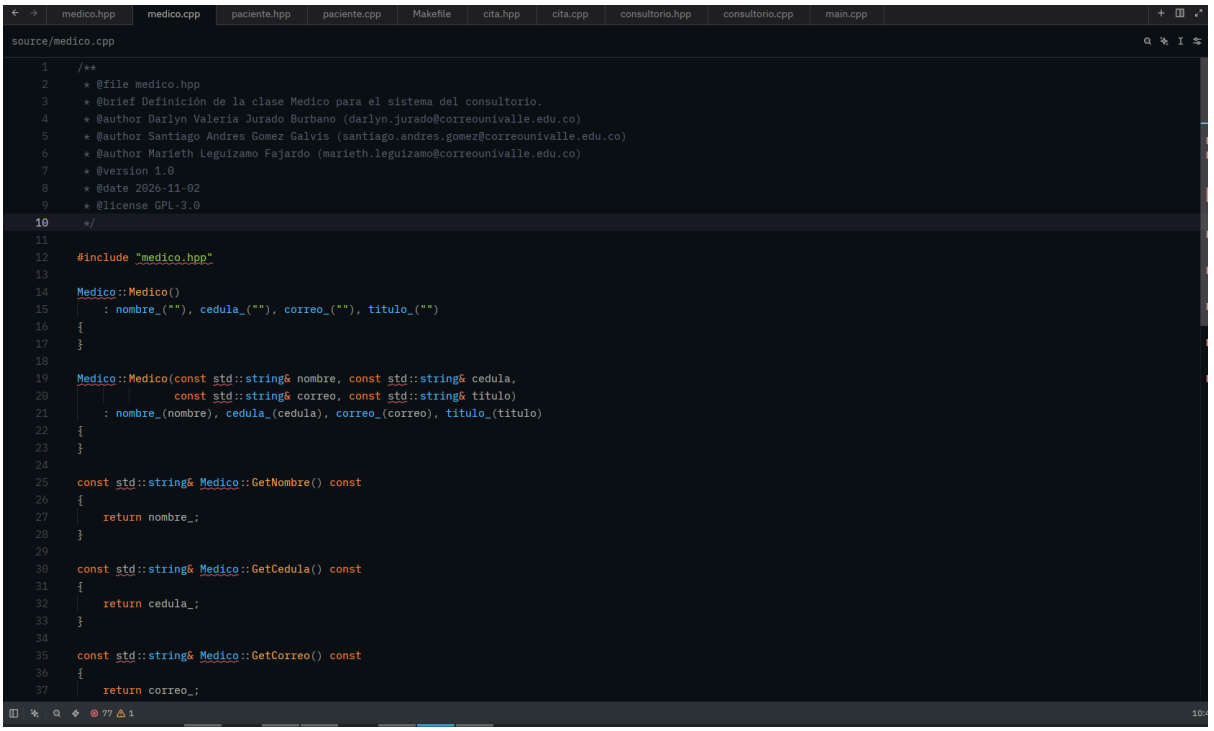
Campo	Contenido a contemplar
Clase	Consultorio
Método	EstaDisponible
Propósito (una línea)	Indica si el médico de la posición dada no tiene ya una cita en esa fecha y hora.
E/P/S	
Entradas (datos requeridos)	posicion(int), fecha(string), hora (string).
Proceso (pasos, decisiones, fórmulas si aplica)	1. Obtener la cédula del médico en posicion. 2. Recorrer las citas registradas. 3. Si alguna tiene la misma cédula de médico, la misma fecha y la misma hora, retornar falso. 4. Si termina el recorrido, retornar verdadero.
Salidas (resultado observable)	bool: Verdadero si esta disponible
Contrato PRE/POST	
PRE	0 <= posicion < TOTAL_MEDICOS
POST	No modifica el estado (const).
Pseudocódigo / Algoritmo	
Algoritmo	Algoritmo EstaDisponible(posicion, fecha, hora) cedula_medico ← ObtenerMedico(posicion).GetCedula() Para i ← 0 Hasta TamañoDe(citas_) - 1 Hacer Si citas_[i].GetMedico().GetCedula() = cedula_medico Y citas_[i].GetFecha() = fecha Y citas_[i].GetHora() = hora Entonces Retornar Falso FinSi FinPara Retornar Verdadero FinAlgoritmo

Campo	Contenido a contemplar
Clase	Consultorio
Método	AgendarCita
Propósito (una línea)	Registra una cita en la agenda si el medico elegido esta disponible.
E/P/S	
Entradas (datos requeridos)	`paciente` (const Paciente&), `posicion` (int), `fecha` (string), `hora` (string).
Proceso (pasos, decisiones, fórmulas si aplica)	1. Verificar disponibilidad. 2. Si no está disponible, retornar falso. 3. Crear la Cita con paciente, médico, fecha y hora. 4. Agregarla a `citas_`. 5. Retornar verdadero.
Salidas (resultado observable)	bool: verdadero si la cita quedó registrada
Contrato PRE/POST	
PRE	0 <= posicion < TOTAL_MEDICOS ; los datos del paciente, la fecha y la hora ya fueron validados.
POST	Si retorna verdadero, `citas_` crece en 1. Si retorna falso, no cambia.
Pseudocódigo / Algoritmo	
Algoritmo	Algoritmo AgendarCita(paciente, posicion, fecha, hora) Si NO EstaDisponible(posicion, fecha, hora) Entonces Retornar Falso FinSi nueva_cita ← Cita(paciente, ObtenerMedico(posicion), fecha, hora) Agregar nueva_cita a citas_ Retornar Verdadero FinAlgoritmo

Campo	Contenido a contemplar
Clase	Consultorio
Método	ObtenerMedico
Propósito (una línea)	Retorna la referencia del médico en la posición solicitada.
E/P/S	
Entradas (datos requeridos)	posicion (int)
Proceso (pasos, decisiones, fórmulas si aplica)	1. Comprobar que `posicion` esté en [0, TOTAL_MEDICOS-1]. 2. Si no, lanzar `std::out_of_range`. 3. Retornar `medicos_[posicion]`.
Salidas (resultado observable)	`const Medico&`.
Contrato PRE/POST	
PRE	0 <= posicion < TOTAL_MEDICOS.
POST	Retorna la referencia sin copias ni cambios de estado (const).
Pseudocódigo / Algoritmo	
Algoritmo	Algoritmo ObtenerMedico(posicion) Si posicion < 0 O posicion >= TOTAL_MEDICOS Entonces LanzarExcepcion("Posicion fuera de rango") FinSi Retornar medicos_[posicion] FinAlgoritmo

6. Programa en c++

medico.cpp



```
/**
 * @file medico.hpp
 * @brief Definición de la clase Medico para el sistema del consultorio.
 * @author Darlyn Valeria Jurado Burbano (darlyn.jurado@correounivalle.edu.co)
 * @author Santiago Andres Gomez Galvis (santiago.andres.gomez@correounivalle.edu.co)
 * @author Marieth Leguizamo Fajardo (marieth.leguizamo@correounivalle.edu.co)
 * @version 1.0
 * @date 2026-11-02
 * @license GPL-3.0
 */

#include "medico.hpp"

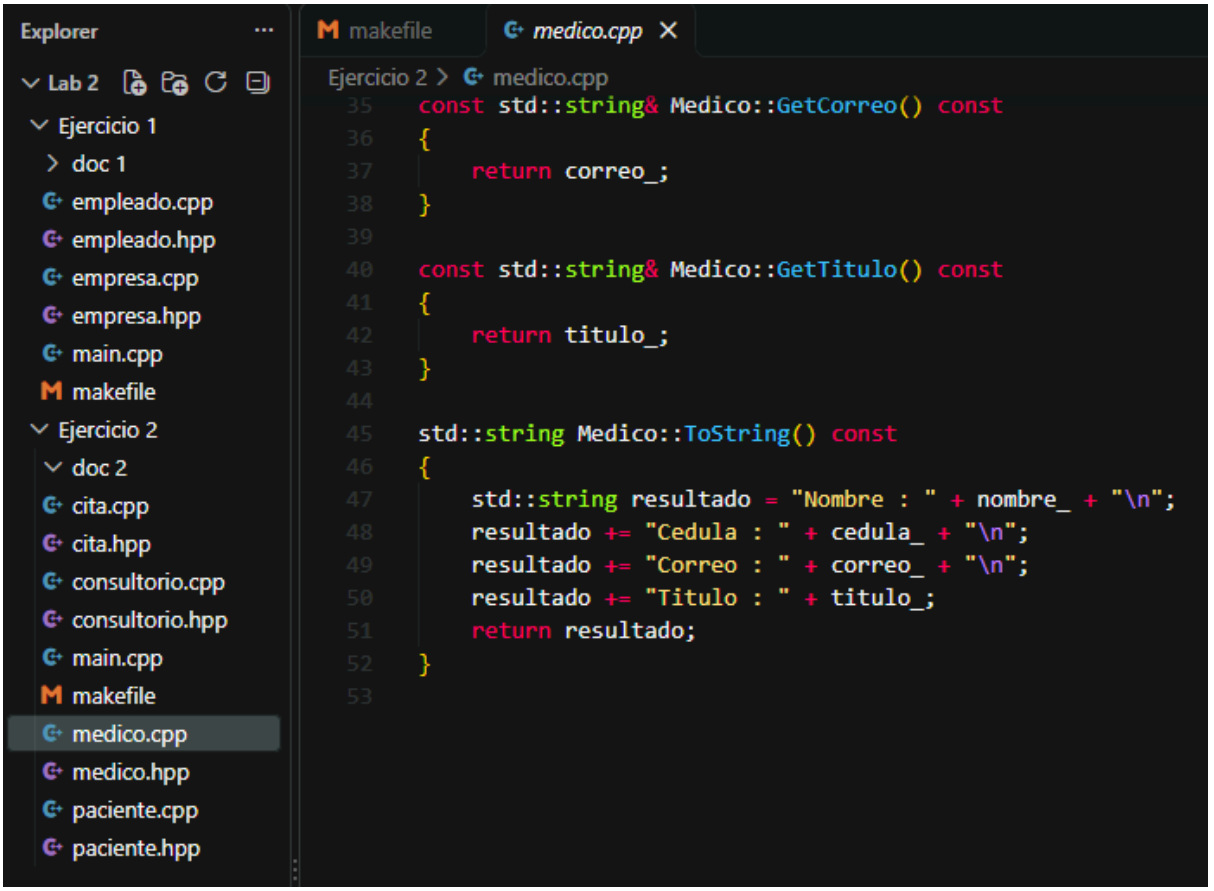
Medico::Medico()
    : nombre_(""), cedula_(""), correo_(""), titulo_("")
{
}

Medico::Medico(const std::string& nombre, const std::string& cedula,
               const std::string& correo, const std::string& titulo)
    : nombre_(nombre), cedula_(cedula), correo_(correo), titulo_(titulo)
{
}

const std::string& Medico::GetNombre() const
{
    return nombre_;
}

const std::string& Medico::GetCedula() const
{
    return cedula_;
}

const std::string& Medico::GetCorreo() const
{
    return correo_;
```

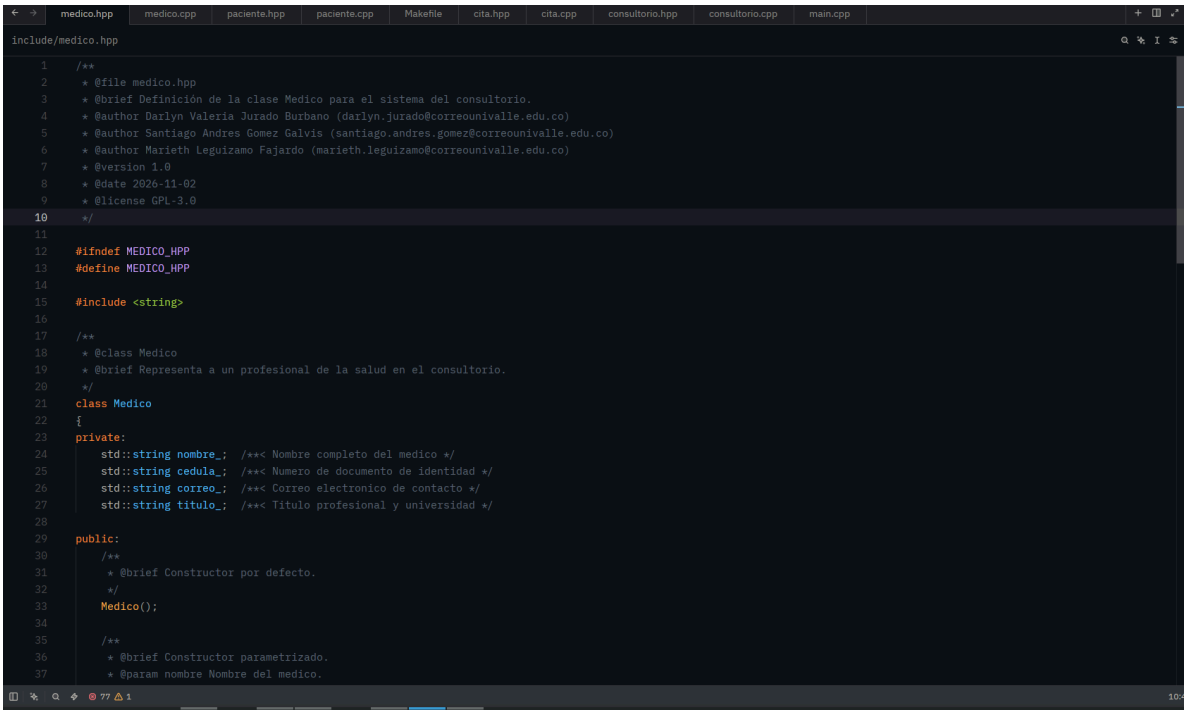


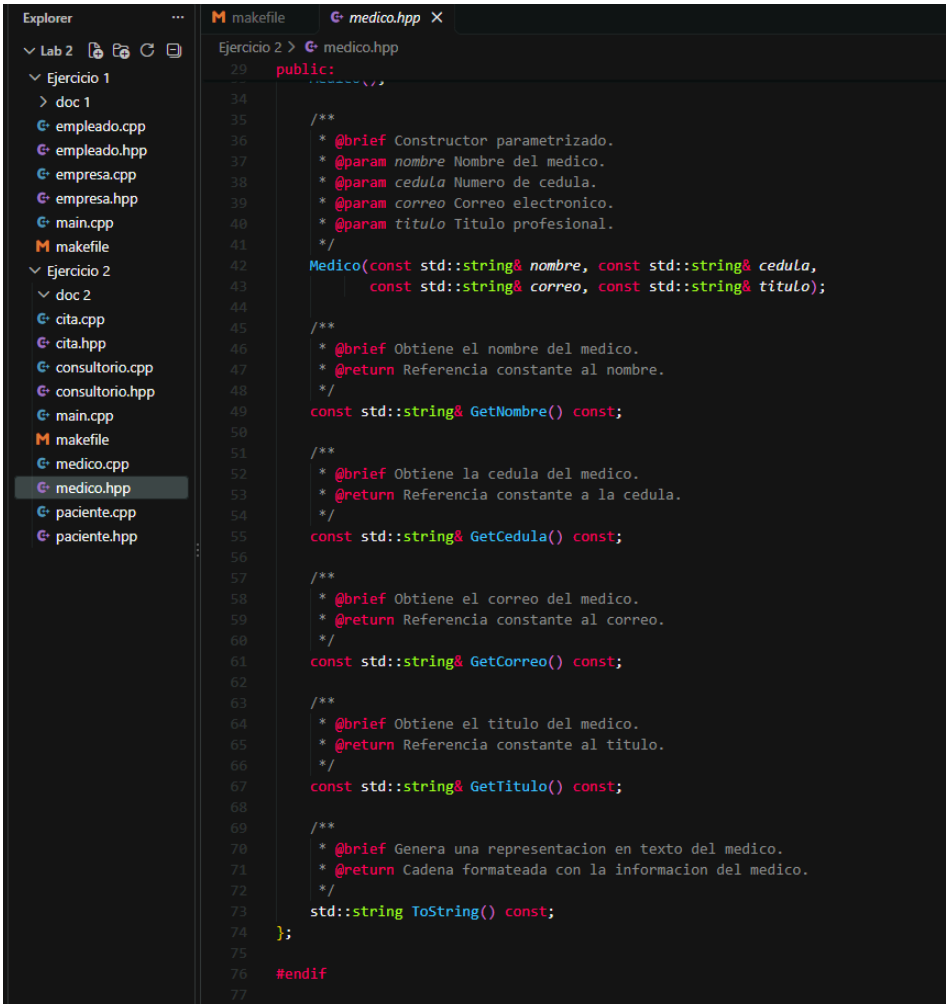
```
const std::string& Medico::GetCorreo() const
{
    return correo_;
}

const std::string& Medico::GetTitulo() const
{
    return titulo_;
}

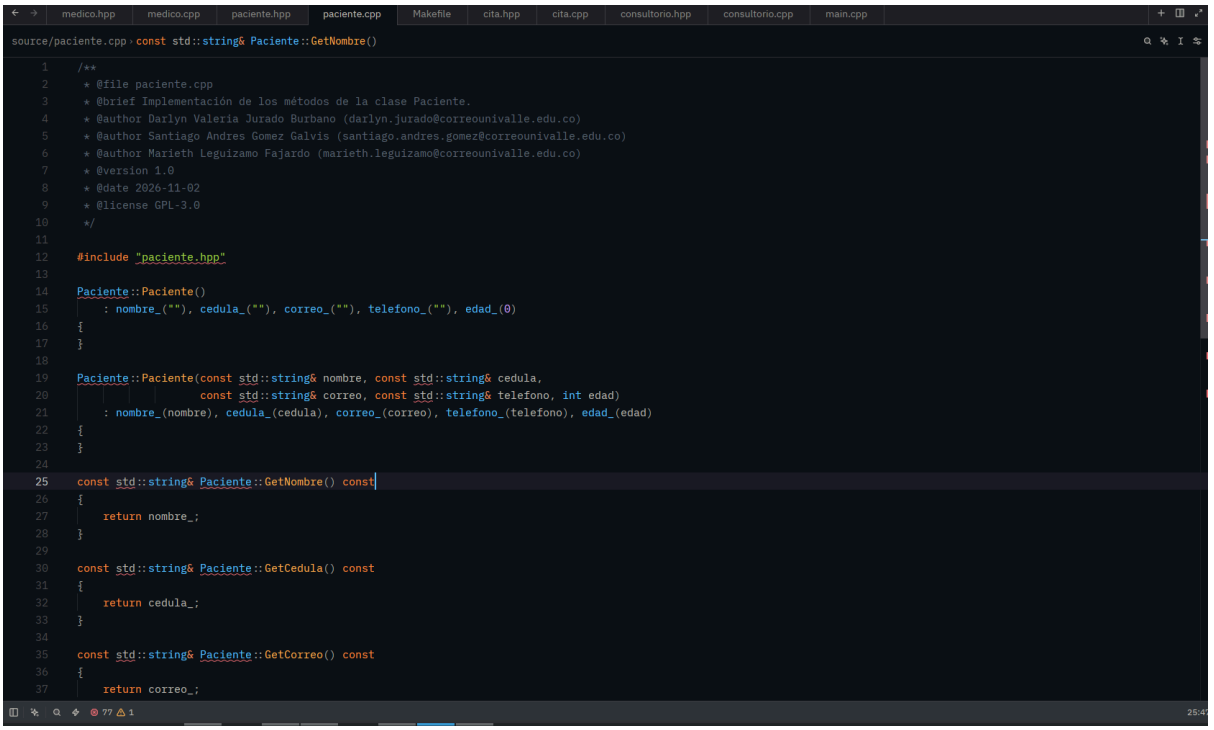
std::string Medico::ToString() const
{
    std::string resultado = "Nombre : " + nombre_ + "\n";
    resultado += "Cedula : " + cedula_ + "\n";
    resultado += "Correo : " + correo_ + "\n";
    resultado += "Titulo : " + titulo_;
    return resultado;
}
```

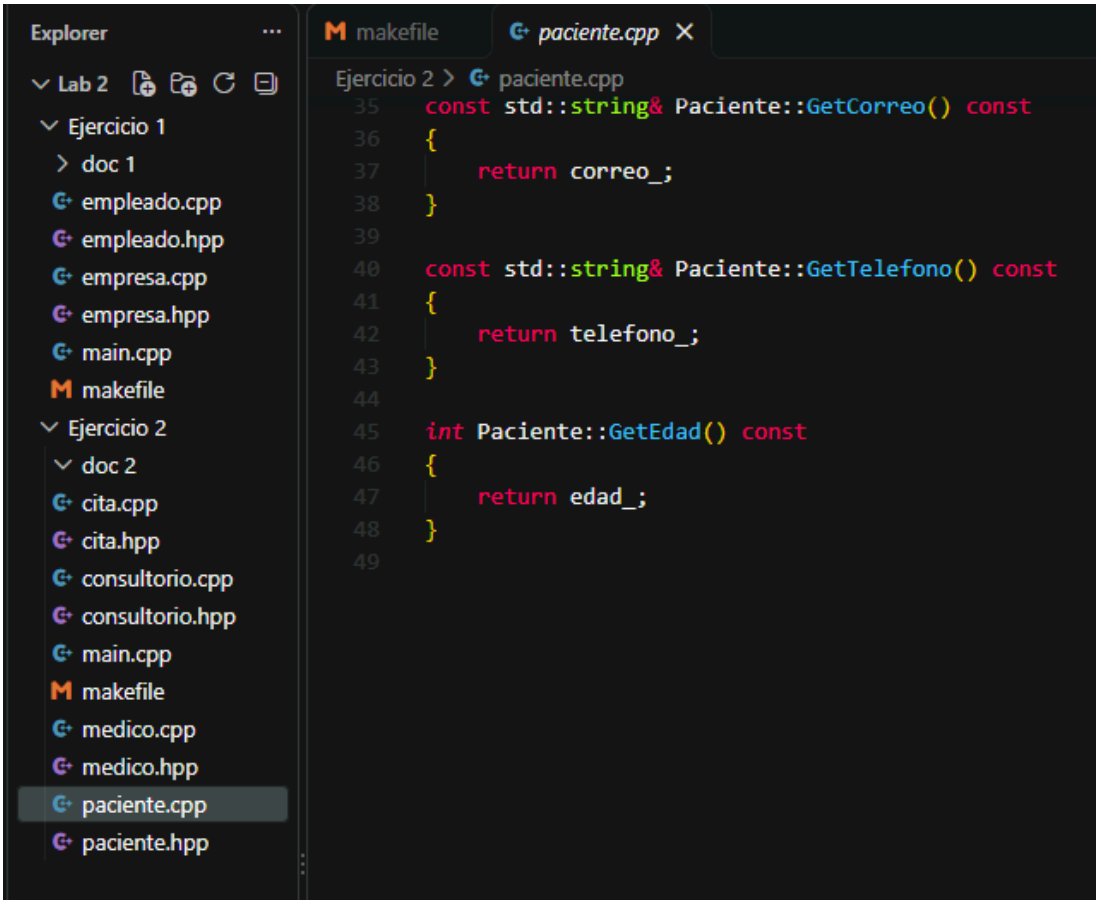
medico.hpp


medico.hpp
medico.cpp
paciente.hpp
paciente.cpp
Makefile
cita.hpp
cita.cpp
consultorio.hpp
consultorio.cpp
main.cpp
include/medico.hpp
/**
* @file medico.hpp
* @brief Definición de la clase Medico para el sistema del consultorio.
* @author Darlyn Valeria Jurado Burbano (darlyn.jurado@correounivalle.edu.co)
* @author Santiago Andres Gomez Galvis (santiago.andres.gomez@correounivalle.edu.co)
* @author Marieth Leguizamo Fajardo (marieth.leguizamo@correounivalle.edu.co)
* @version 1.0
* @date 2026-11-02
* @license GPL-3.0
*/
#ifndef MEDICO_HPP
#define MEDICO_HPP
#include <string>
/**
* @class Medico
* @brief Representa a un profesional de la salud en el consultorio.
*/
class Medico
{
private:
std::string nombre_; /**< Nombre completo del medico */
std::string cedula_; /**< Numero de documento de identidad */
std::string correo_; /**< Correo electronico de contacto */
std::string titulo_; /**< Titulo profesional y universidad */
public:
/**
* @brief Constructor por defecto.
*/
Medico();
/**
* @brief Constructor parametrizado.
* @param nombre Nombre del medico.

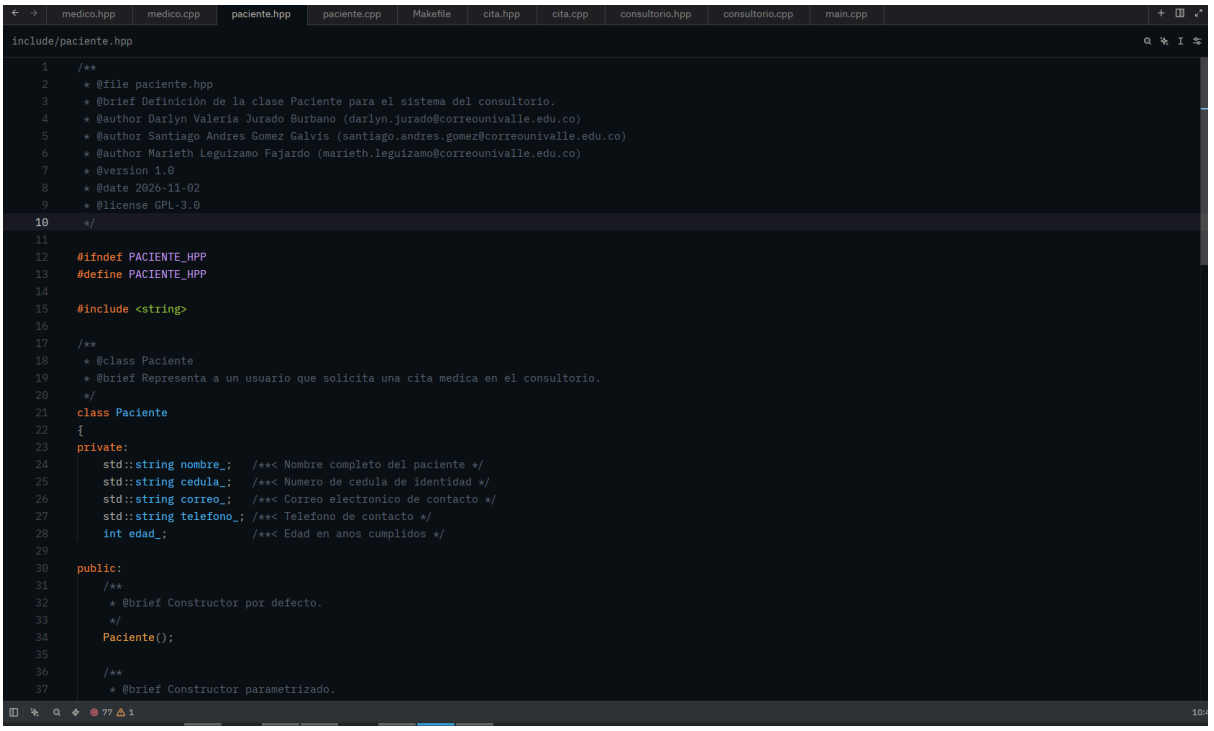

Explorer
Lab 2
Ejercicio 1
doc 1
empleado.cpp
empleado.hpp
empresa.cpp
empresa.hpp
main.cpp
makefile
Ejercicio 2
doc 2
cita.cpp
cita.hpp
consultorio.cpp
consultorio.hpp
main.cpp
makefile
medico.cpp
medico.hpp
paciente.cpp
paciente.hpp
makefile
medico.hpp
Ejercicio 2 > medico.hpp
public:
/**
* @brief Constructor parametrizado.
* @param nombre Nombre del medico.
* @param cedula Numero de cedula.
* @param correo Correo electronico.
* @param titulo Titulo profesional.
*/
Medico(const std::string& nombre, const std::string& cedula,
const std::string& correo, const std::string& titulo);
/**
* @brief Obtiene el nombre del medico.
* @return Referencia constante al nombre.
*/
const std::string& GetNombre() const;
/**
* @brief Obtiene la cedula del medico.
* @return Referencia constante a la cedula.
*/
const std::string& GetCedula() const;
/**
* @brief Obtiene el correo del medico.
* @return Referencia constante al correo.
*/
const std::string& GetCorreo() const;
/**
* @brief Obtiene el titulo del medico.
* @return Referencia constante al titulo.
*/
const std::string& GetTitulo() const;
/**
* @brief Genera una representacion en texto del medico.
* @return Cadena formateada con la informacion del medico.
*/
std::string ToString() const;
};
#endif

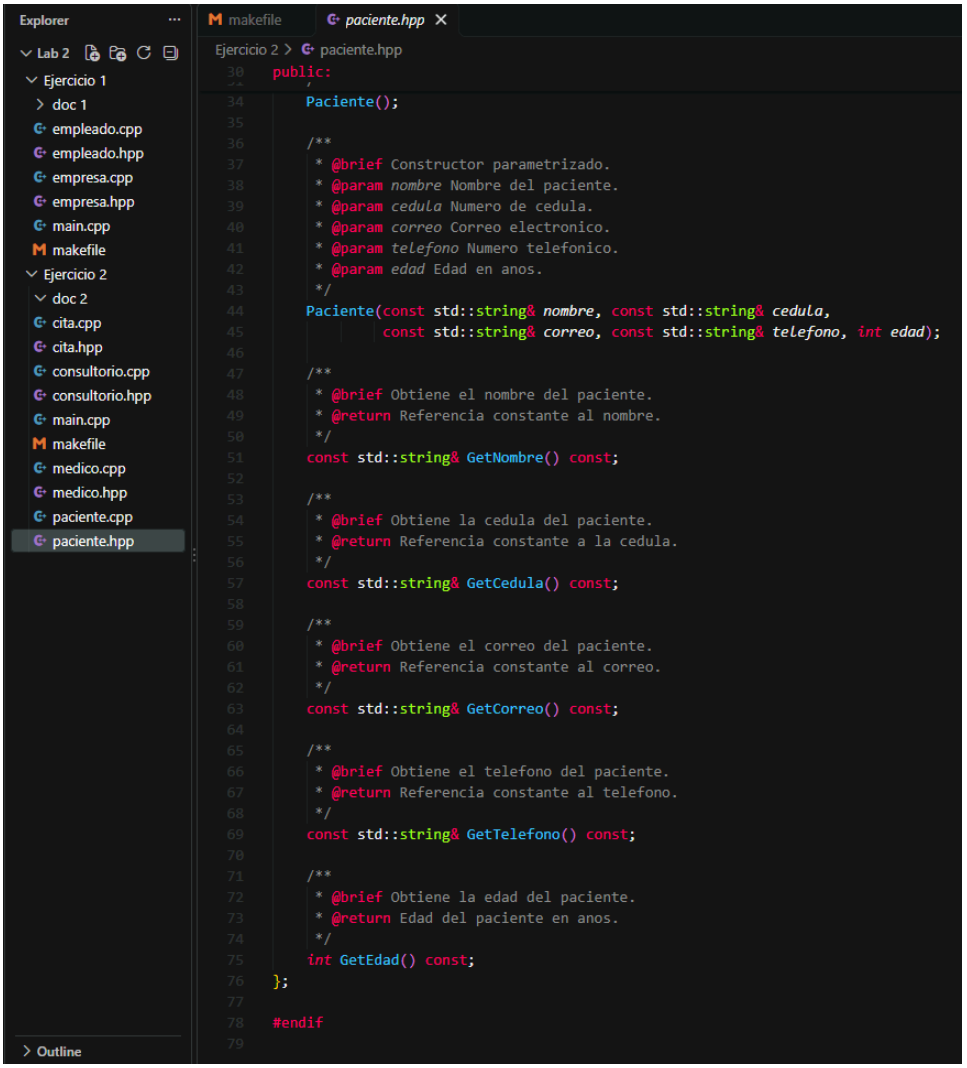
paciente.cpp



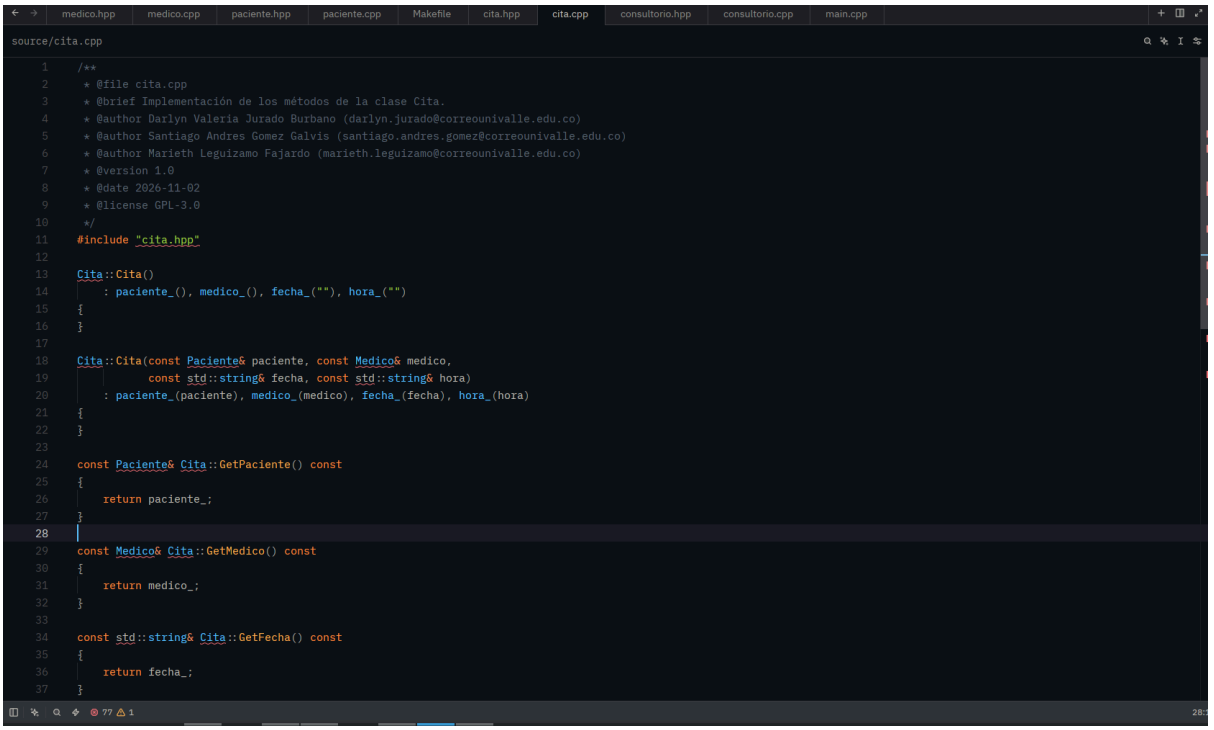


paciente.hpp





cita.cpp



```cpp
/**
 * @file cita.cpp
 * @brief Implementación de los métodos de la clase Cita.
 * @author Darlyn Valeria Jurado Burbano (darlyn.jurado@correounivalle.edu.co)
 * @author Santiago Andres Gomez Galvis (santiago.andres.gomez@correounivalle.edu.co)
 * @author Marieth Leguizamo Fajardo (marieth.leguizamo@correounivalle.edu.co)
 * @version 1.0
 * @date 2026-11-02
 * @license GPL-3.0
 */
#include "cita.hpp"

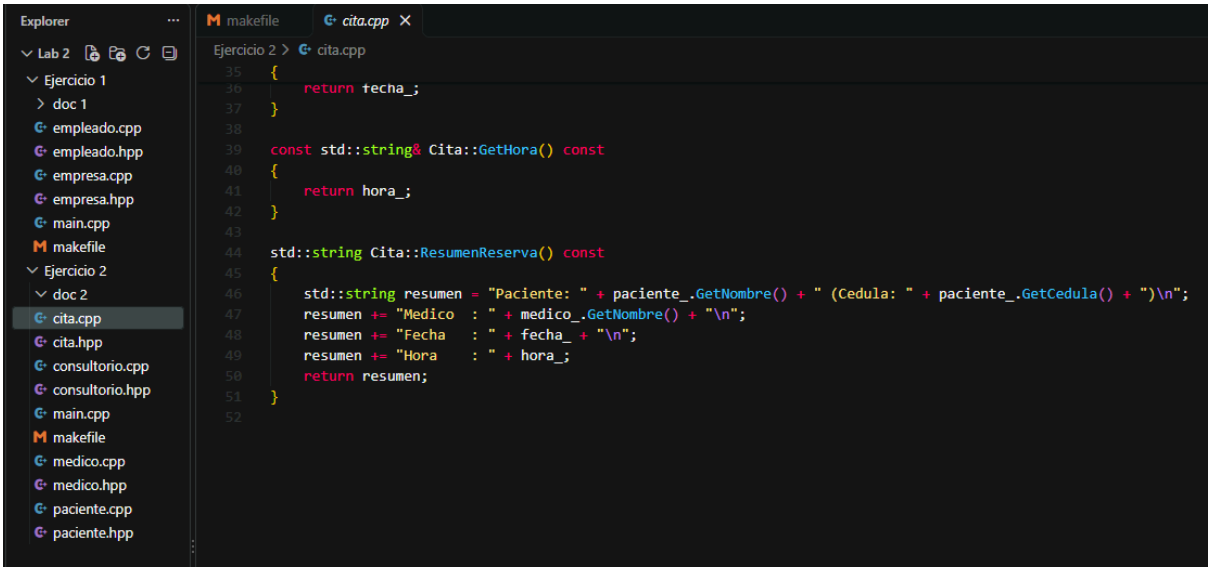
Cita::Cita()
    : paciente_(), medico_(), fecha_(""), hora_("")
{
}

Cita::Cita(const Paciente& paciente, const Medico& medico,
           const std::string& fecha, const std::string& hora)
    : paciente_(paciente), medico_(medico), fecha_(fecha), hora_(hora)
{
}

const Paciente& Cita::GetPaciente() const
{
    return paciente_;
}

const Medico& Cita::GetMedico() const
{
    return medico_;
}

const std::string& Cita::GetFecha() const
{
    return fecha_;
}
```

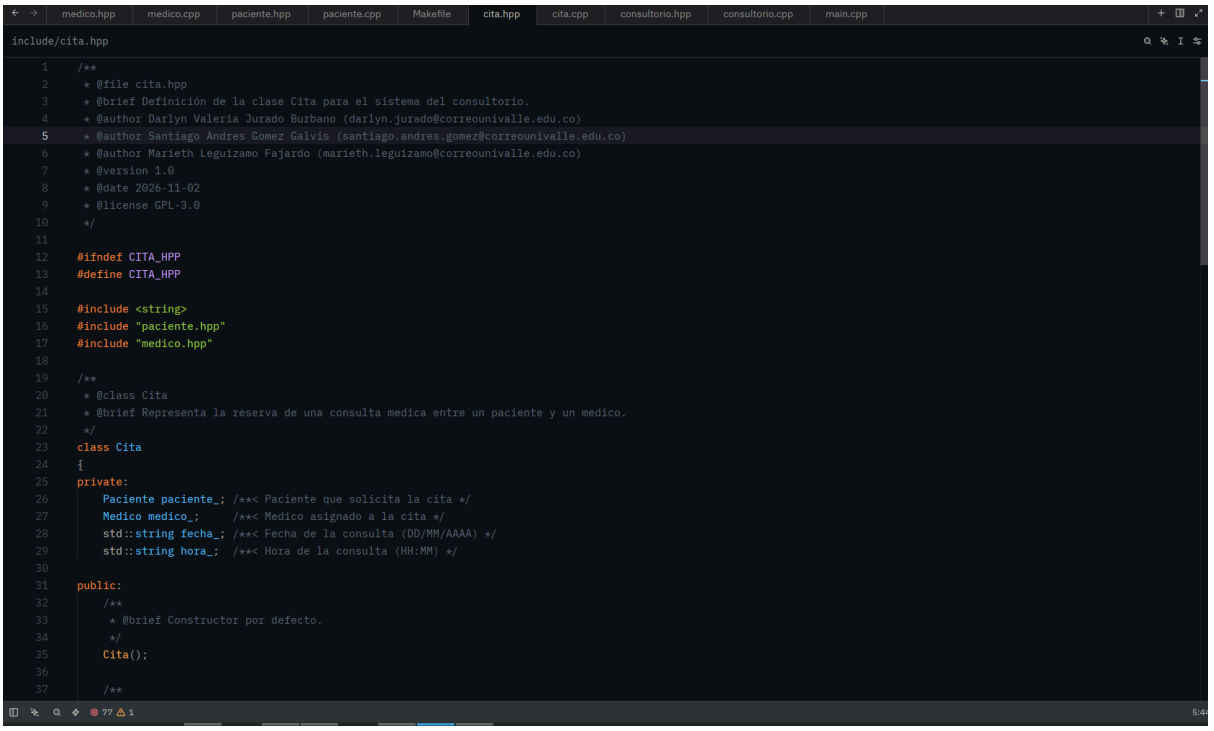


```cpp
{
    return fecha_;
}

const std::string& Cita::GetHora() const
{
    return hora_;
}

std::string Cita::ResumenReserva() const
{
    std::string resumen = "Paciente: " + paciente_.GetNombre() + " (Cedula: " + paciente_.GetCedula() + ")\n";
    resumen += "Medico  : " + medico_.GetNombre() + "\n";
    resumen += "Fecha   : " + fecha_ + "\n";
    resumen += "Hora    : " + hora_;
    return resumen;
}
```

cita.hpp



```
include/cita.hpp
```

```cpp
/**
 * @file cita.hpp
 * @brief Definición de la clase Cita para el sistema del consultorio.
 * @author Darlyn Valeria Jurado Burbano (darlyn.jurado@correounivalle.edu.co)
 * @author Santiago Andres Gomez Galvis (santiago.andres.gomez@correounivalle.edu.co)
 * @author Marieth Leguizamo Fajardo (marieth.leguizamo@correounivalle.edu.co)
 * @version 1.0
 * @date 2026-11-02
 * @license GPL-3.0
 */

#ifndef CITA_HPP
#define CITA_HPP

#include <string>
#include "paciente.hpp"
#include "medico.hpp"

/**
 * @class Cita
 * @brief Representa la reserva de una consulta medica entre un paciente y un medico.
 */
class Cita
{
private:
    Paciente paciente_; /**< Paciente que solicita la cita */
    Medico medico_;     /**< Medico asignado a la cita */
    std::string fecha_; /**< Fecha de la consulta (DD/MM/AAAA) */
    std::string hora_;  /**< Hora de la consulta (HH:MM) */

public:
    /**
     * @brief Constructor por defecto.
     */
    Cita();

    /**
```



```
Explorer
Lab 2
  Ejercicio 1
    doc 1
    empleado.cpp
    empleado.hpp
    empresa.cpp
    empresa.hpp
    main.cpp
    makefile
  Ejercicio 2
    doc 2
    cita.cpp
    cita.hpp
    consultorio.cpp
    consultorio.hpp
    main.cpp
    makefile
    medico.cpp
    medico.hpp
    paciente.cpp
    paciente.hpp
```

```
makefile   cita.hpp
Ejercicio 2 > cita.hpp
```

```cpp
public:

    /**
     * @brief Constructor parametrizado.
     * @param paciente Objeto paciente.
     * @param medico Objeto medico.
     * @param fecha Cadena con la fecha.
     * @param hora Cadena con la hora.
     */
    Cita(const Paciente& paciente, const Medico& medico,
         const std::string& fecha, const std::string& hora);

    /**
     * @brief Obtiene el paciente asignado.
     * @return Referencia constante al paciente.
     */
    const Paciente& GetPaciente() const;

    /**
     * @brief Obtiene el medico asignado.
     * @return Referencia constante al medico.
     */
    const Medico& GetMedico() const;

    /**
     * @brief Obtiene la fecha de la cita.
     * @return Referencia constante a la fecha.
     */
    const std::string& GetFecha() const;

    /**
     * @brief Obtiene la hora de la cita.
     * @return Referencia constante a la hora.
     */
    const std::string& GetHora() const;

    /**
     * @brief Genera el resumen estructurado de la reserva.
     * @return Cadena formateada con los datos de la cita.
     */
    std::string ResumenReserva() const;
};

#endif
```

consultorio.cpp

include/consultorio.hpp › class Consultorio

```hpp
/**
 * @file consultorio.hpp
 * @brief Definición de la clase Consultorio para la gestión de médicos y citas.
 * @author Darlyn Valeria Jurado Burbano (darlyn.jurado@correounivalle.edu.co)
 * @author Santiago Andres Gomez Galvis (santiago.andres.gomez@correounivalle.edu.co)
 * @author Marieth Leguizamo Fajardo (marieth.leguizamo@correounivalle.edu.co)
 * @version 1.0
 * @date 2026-11-02
 * @license GPL-3.0
 */

#ifndef CONSULTORIO_HPP
#define CONSULTORIO_HPP

#include <string>
#include <vector>
#include "medico.hpp"
#include "paciente.hpp"
#include "cita.hpp"

/**
 * @class Consultorio
 * @brief Gestiona los medicos del establecimiento y la agenda general de citas.
 */
class Consultorio
{
public:
    static const int TOTAL_MEDICOS = 3; /**< Numero fijo de medicos en el consultorio */

private:
    std::string nombre_;              /**< Nombre del consultorio */
    Medico medicos_[TOTAL_MEDICOS];   /**< Arreglo fijo de medicos */
    std::vector<Cita> citas_;         /**< Agenda de citas registradas */

public:
    /**
     * @brief Constructor parametrizado.
```

Explorer

Lab 2

- Ejercicio 1
  - doc 1
  - empleado.cpp
  - empleado.hpp
  - empresa.cpp
  - empresa.hpp
  - main.cpp
  - makefile
- Ejercicio 2
  - doc 2
  - cita.cpp
  - cita.hpp
  - consultorio.cpp
  - consultorio.hpp
  - main.cpp
  - makefile
  - medico.cpp
  - medico.hpp
  - paciente.cpp
  - paciente.hpp

makefile | consultorio.cpp

Ejercicio 2 › consultorio.cpp

```cpp
const Medico& Consultorio::ObtenerMedico(int posicion) const
{
    if (posicion < 0 || posicion >= TOTAL_MEDICOS)
    {
        throw std::out_of_range("Posicion fuera de rango");
    }
    return medicos_[posicion];
}

bool Consultorio::EstaDisponible(int posicion, const std::string& fecha, const std::string& hora) const
{
    const Medico& medico_buscado = ObtenerMedico(posicion);

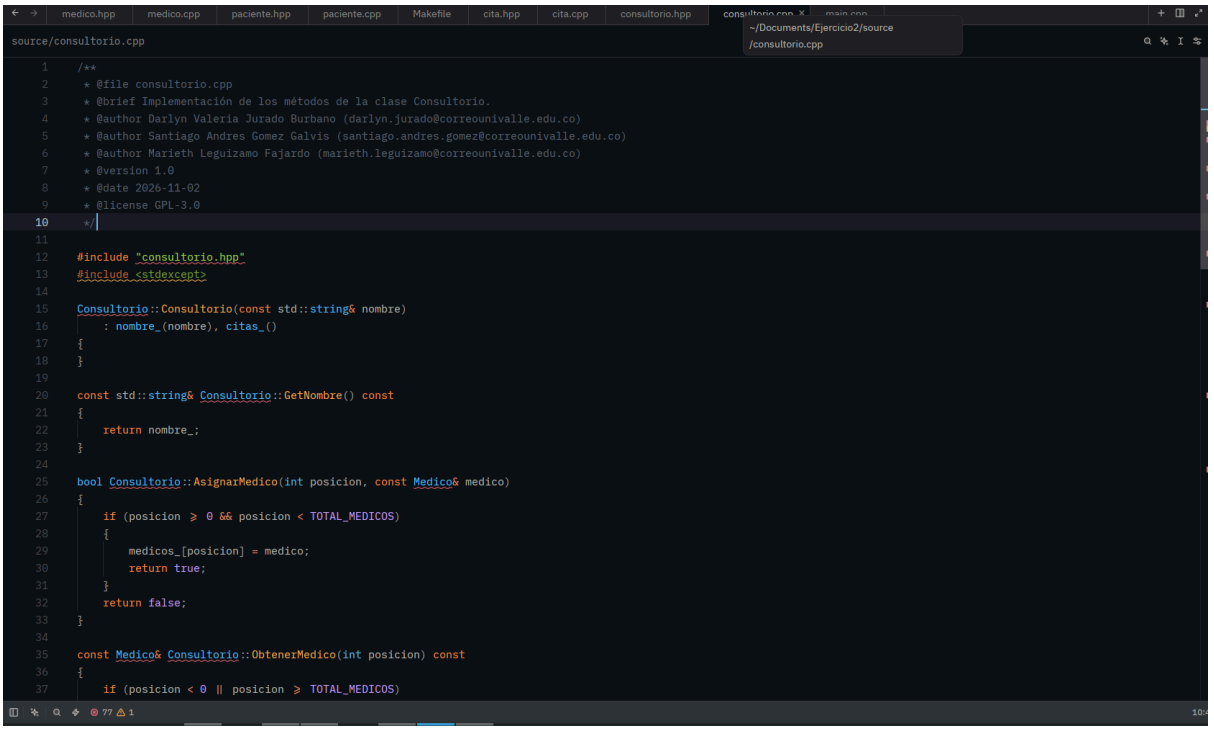
    for (std::size_t i = 0; i < citas_.size(); ++i)
    {
        if (citas_[i].GetMedico().GetCedula() == medico_buscado.GetCedula() &&
            citas_[i].GetFecha() == fecha &&
            citas_[i].GetHora() == hora)
        {
            return false;
        }
    }
    return true;
}

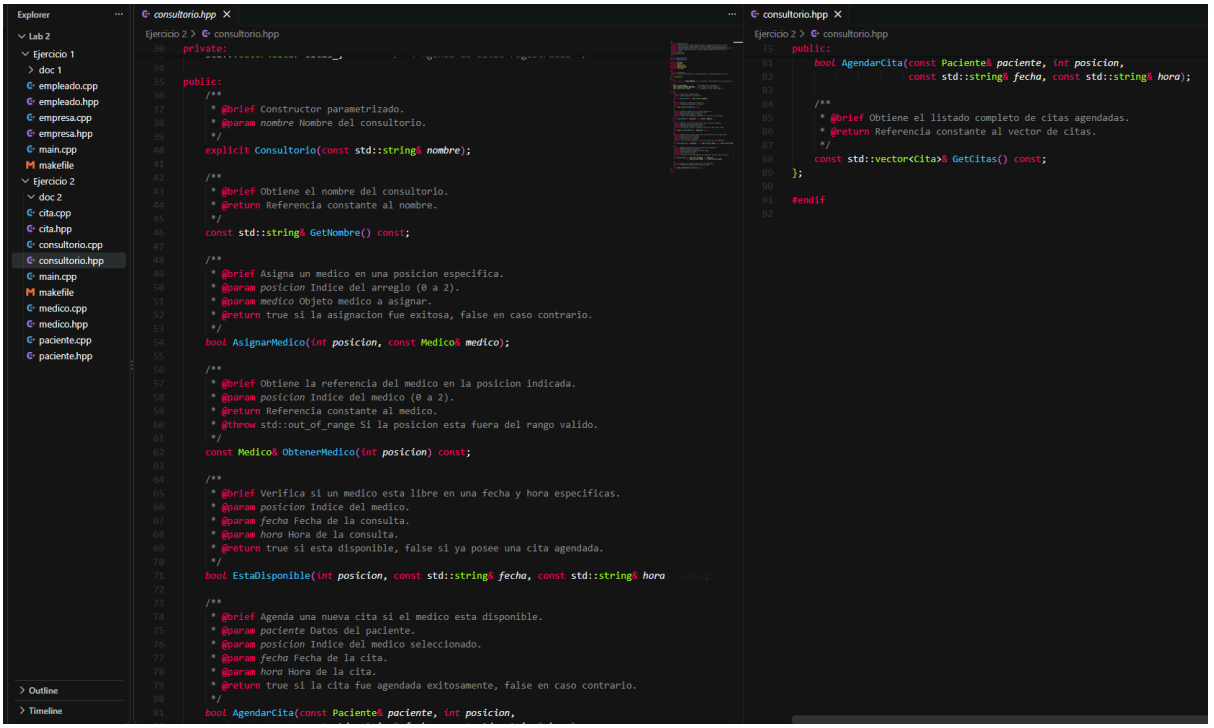
bool Consultorio::AgendarCita(const Paciente& paciente, int posicion,
                              const std::string& fecha, const std::string& hora)
{
    if (!EstaDisponible(posicion, fecha, hora))
    {
        return false;
    }

    Cita nueva_cita(paciente, ObtenerMedico(posicion), fecha, hora);
    citas_.push_back(nueva_cita);
    return true;
}

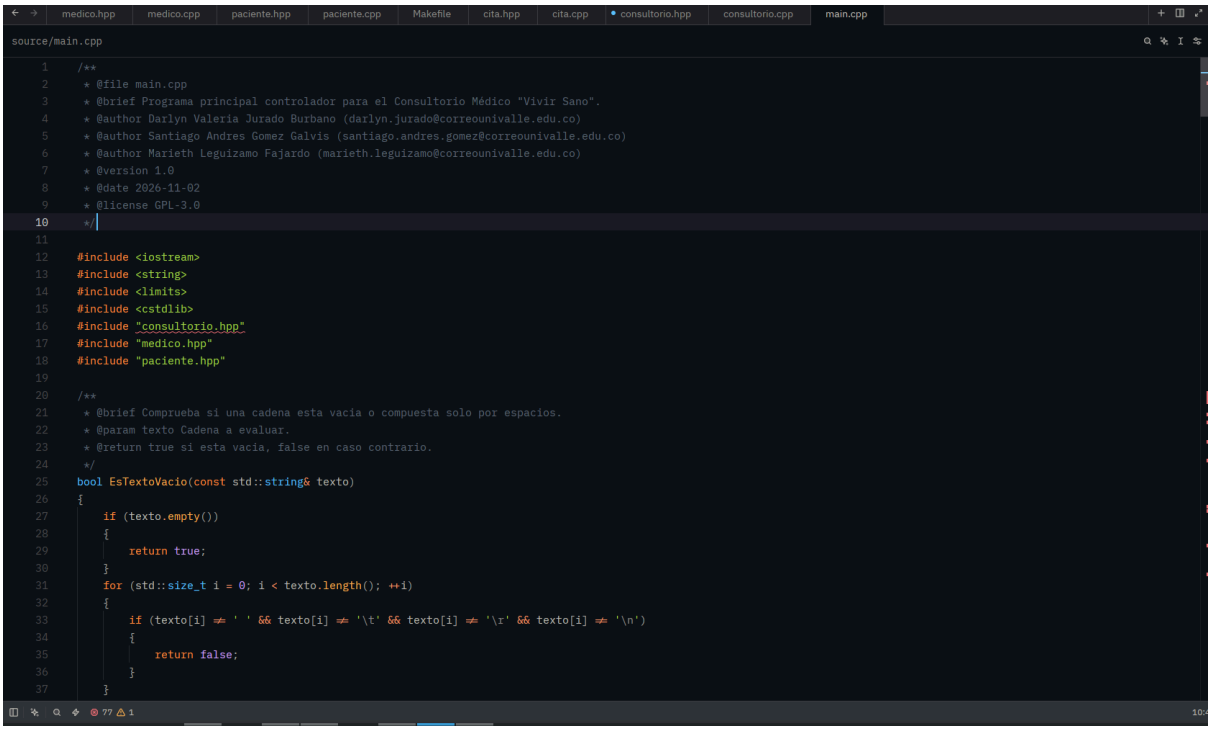
const std::vector<Cita>& Consultorio::GetCitas() const
{
    return citas_;
}
```

consultorio.hpp





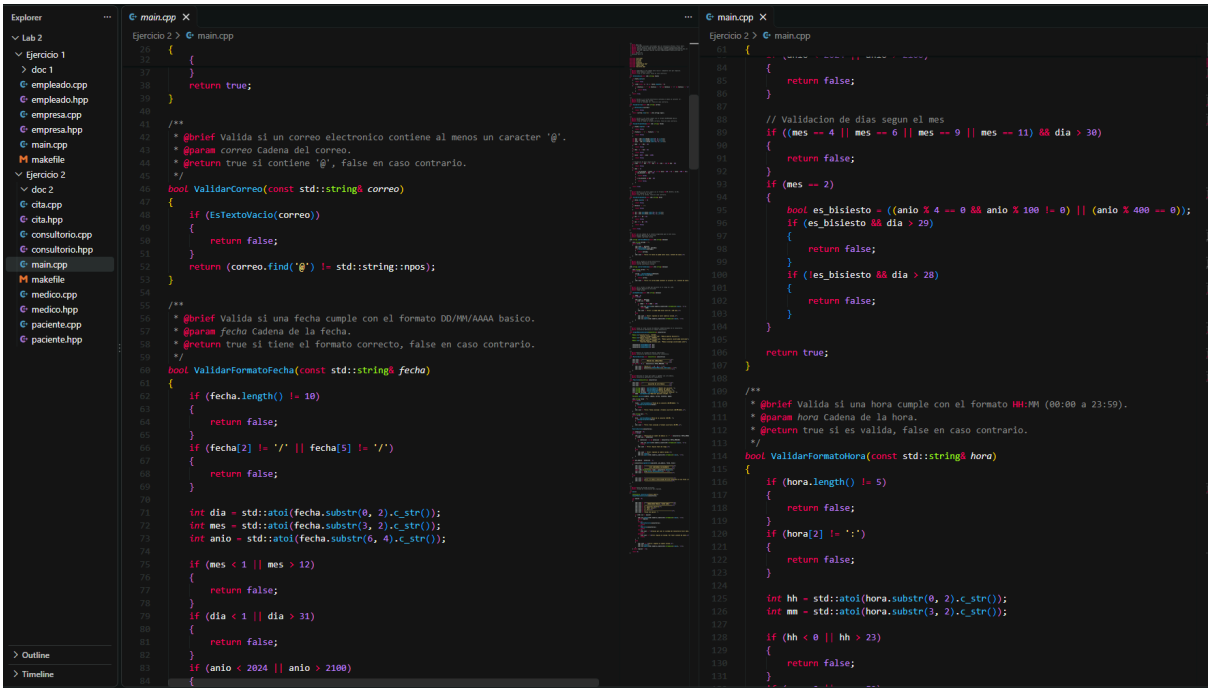
main.cpp



```cpp
/**
 * @file main.cpp
 * @brief Programa principal controlador para el Consultorio Médico "Vivir Sano".
 * @author Darlyn Valeria Jurado Burbano (darlyn.jurado@correounivalle.edu.co)
 * @author Santiago Andres Gomez Galvis (santiago.andres.gomez@correounivalle.edu.co)
 * @author Marieth Leguizamo Fajardo (marieth.leguizamo@correounivalle.edu.co)
 * @version 1.0
 * @date 2026-11-02
 * @license GPL-3.0
 */

#include <iostream>
#include <string>
#include <limits>
#include <cstdlib>
#include "consultorio.hpp"
#include "medico.hpp"
#include "paciente.hpp"

/**
 * @brief Comprueba si una cadena esta vacia o compuesta solo por espacios.
 * @param texto Cadena a evaluar.
 * @return true si esta vacia, false en caso contrario.
 */
bool EsTextoVacio(const std::string& texto)
{
    if (texto.empty())
    {
        return true;
    }
    for (std::size_t i = 0; i < texto.length(); ++i)
    {
        if (texto[i] != ' ' && texto[i] != '\t' && texto[i] != '\r' && texto[i] != '\n')
        {
            return false;
        }
    }
```



```cpp
/**
 * @brief Valida si un correo electronico contiene al menos un caracter '@'.
 * @param correo Cadena del correo.
 * @return true si contiene '@', false en caso contrario.
 */
bool ValidarCorreo(const std::string& correo)
{
    if (EsTextoVacio(correo))
    {
        return false;
    }
    return (correo.find('@') != std::string::npos);
}

/**
 * @brief Valida si una fecha cumple con el formato DD/MM/AAAA basico.
 * @param fecha Cadena de la fecha.
 * @return true si tiene el formato correcto, false en caso contrario.
 */
bool ValidarFormatoFecha(const std::string& fecha)
{
    if (fecha.length() != 10)
    {
        return false;
    }
    if (fecha[2] != '/' || fecha[5] != '/')
    {
        return false;
    }

    int dia = std::atoi(fecha.substr(0, 2).c_str());
    int mes = std::atoi(fecha.substr(3, 2).c_str());
    int anio = std::atoi(fecha.substr(6, 4).c_str());

    if (mes < 1 || mes > 12)
    {
        return false;
    }
    if (dia < 1 || dia > 31)
    {
        return false;
    }
    if (anio < 2024 || anio > 2100)
    {
```

```cpp
    {
        return false;
    }

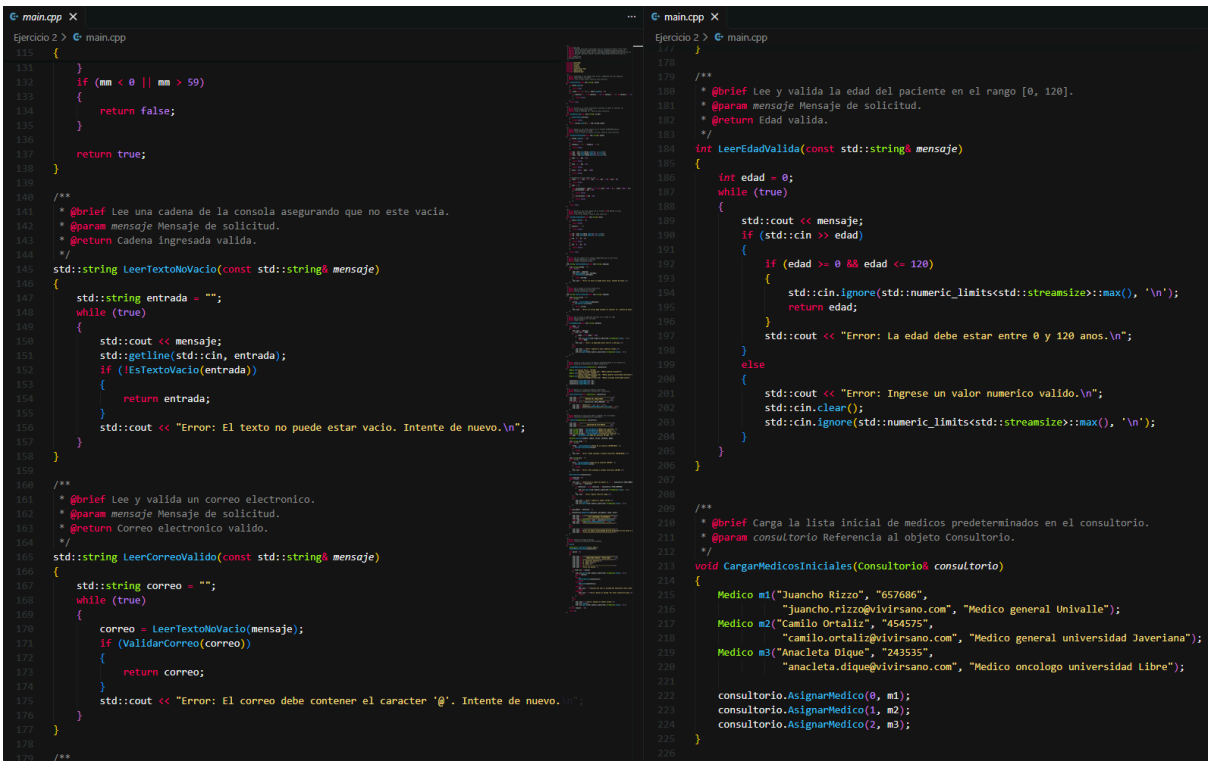
    // Validacion de dias segun el mes
    if ((mes == 4 || mes == 6 || mes == 9 || mes == 11) && dia > 30)
    {
        return false;
    }
    if (mes == 2)
    {
        bool es_bisiesto = ((anio % 4 == 0 && anio % 100 != 0) || (anio % 400 == 0));
        if (es_bisiesto && dia > 29)
        {
            return false;
        }
        if (!es_bisiesto && dia > 28)
        {
            return false;
        }
    }

    return true;
}

/**
 * @brief Valida si una hora cumple con el formato HH:MM (00:00 a 23:59).
 * @param hora Cadena de la hora.
 * @return true si es valida, false en caso contrario.
 */
bool ValidarFormatoHora(const std::string& hora)
{
    if (hora.length() != 5)
    {
        return false;
    }
    if (hora[2] != ':')
    {
        return false;
    }

    int hh = std::atoi(hora.substr(0, 2).c_str());
    int mm = std::atoi(hora.substr(3, 2).c_str());

    if (hh < 0 || hh > 23)
    {
        return false;
    }
```



```cpp
{
    }
    if (mm < 0 || mm > 59)
    {
        return false;
    }

    return true;
}

/**
 * @brief Lee una cadena de la consola asegurando que no este vacía.
 * @param mensaje Mensaje de solicitud.
 * @return Cadena ingresada valida.
 */
std::string LeerTextoNoVacio(const std::string& mensaje)
{
    std::string entrada = "";
    while (true)
    {
        std::cout << mensaje;
        std::getline(std::cin, entrada);
        if (!EsTextoVacio(entrada))
        {
            return entrada;
        }
        std::cout << "Error: El texto no puede estar vacio. Intente de nuevo.\n";
    }
}

/**
 * @brief Lee y valida un correo electronico.
 * @param mensaje Mensaje de solicitud.
 * @return Correo electronico valido.
 */
std::string LeerCorreoValido(const std::string& mensaje)
{
    std::string correo = "";
    while (true)
    {
        correo = LeerTextoNoVacio(mensaje);
        if (ValidarCorreo(correo))
        {
            return correo;
        }
        std::cout << "Error: El correo debe contener el caracter '@'. Intente de nuevo.
    }
}

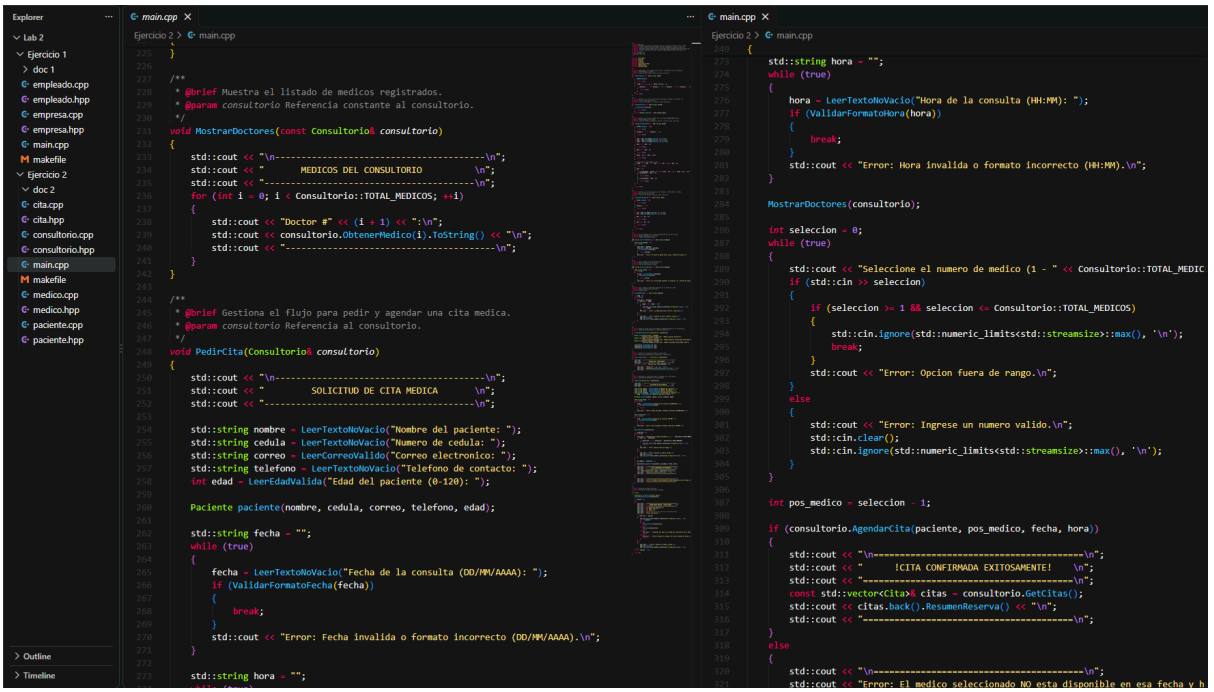
/**
```

```cpp
}

/**
 * @brief Lee y valida la edad del paciente en el rango [0, 120].
 * @param mensaje Mensaje de solicitud.
 * @return Edad valida.
 */
int LeerEdadValida(const std::string& mensaje)
{
    int edad = 0;
    while (true)
    {
        std::cout << mensaje;
        if (std::cin >> edad)
        {
            if (edad >= 0 && edad <= 120)
            {
                std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
                return edad;
            }
            std::cout << "Error: La edad debe estar entre 0 y 120 anos.\n";
        }
        else
        {
            std::cout << "Error: Ingrese un valor numerico valido.\n";
            std::cin.clear();
            std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
        }
    }
}


/**
 * @brief Carga la lista inicial de medicos predeterminados en el consultorio.
 * @param consultorio Referencia al objeto Consultorio.
 */
void CargarMedicosIniciales(Consultorio& consultorio)
{
    Medico m1("Juancho Rizzo", "657686",
              "juancho.rizzo@vivirsano.com", "Medico general Univalle");
    Medico m2("Camilo Ortaliz", "454575",
              "camilo.ortaliz@vivirsano.com", "Medico general universidad Javeriana");
    Medico m3("Anacleta Dique", "243535",
              "anacleta.dique@vivirsano.com", "Medico oncologo universidad Libre");

    consultorio.AsignarMedico(0, m1);
    consultorio.AsignarMedico(1, m2);
    consultorio.AsignarMedico(2, m3);
}
```



```cpp
}

/**
 * @brief Muestra el listado de medicos registrados.
 * @param consultorio Referencia constante al consultorio.
 */
void MostrarDoctores(const Consultorio& consultorio)
{
    std::cout << "\n----------------------------------------\n";
    std::cout << "       MEDICOS DEL CONSULTORIO          \n";
    std::cout << "----------------------------------------\n";
    for (int i = 0; i < Consultorio::TOTAL_MEDICOS; ++i)
    {
        std::cout << "Doctor #" << (i + 1) << ":\n";
        std::cout << consultorio.ObtenerMedico(i).ToString() << "\n";
        std::cout << "----------------------------------------\n";
    }
}

/**
 * @brief Gestiona el flujo para pedir y agendar una cita medica.
 * @param consultorio Referencia al consultorio.
 */
void PedirCita(Consultorio& consultorio)
{
    std::cout << "\n----------------------------------------\n";
    std::cout << "         SOLICITUD DE CITA MEDICA       \n";
    std::cout << "----------------------------------------\n";

    std::string nombre = LeerTextoNoVacio("Nombre del paciente: ");
    std::string cedula = LeerTextoNoVacio("Numero de cedula: ");
    std::string correo = LeerCorreoValido("Correo electronico: ");
    std::string telefono = LeerTextoNoVacio("Telefono de contacto: ");
    int edad = LeerEdadValida("Edad del paciente (0-120): ");

    Paciente paciente(nombre, cedula, correo, telefono, edad);

    std::string fecha = "";
    while (true)
    {
        fecha = LeerTextoNoVacio("Fecha de la consulta (DD/MM/AAAA): ");
        if (ValidarFormatoFecha(fecha))
        {
            break;
        }
        std::cout << "Error: Fecha invalida o formato incorrecto (DD/MM/AAAA).\n";
    }

    std::string hora = "";
```

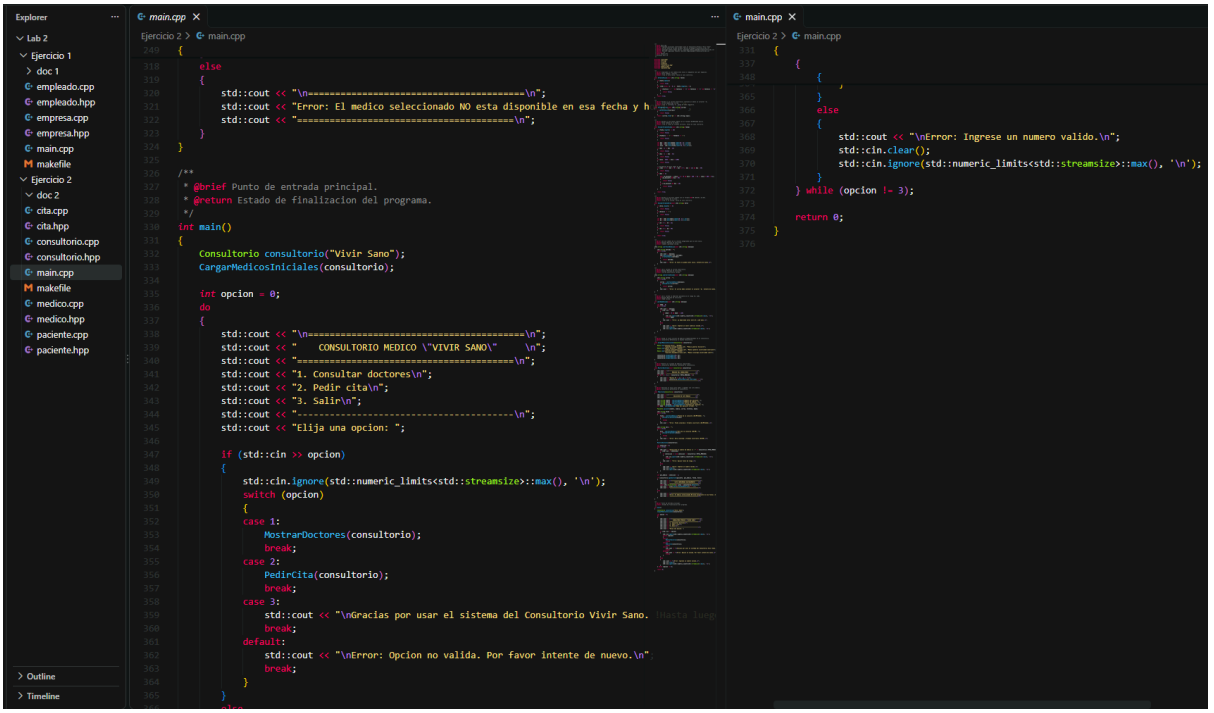
```cpp
{
    std::string hora = "";
    while (true)
    {
        hora = LeerTextoNoVacio("Hora de la consulta (HH:MM): ");
        if (ValidarFormatoHora(hora))
        {
            break;
        }
        std::cout << "Error: Hora invalida o formato incorrecto (HH:MM).\n";
    }

    MostrarDoctores(consultorio);

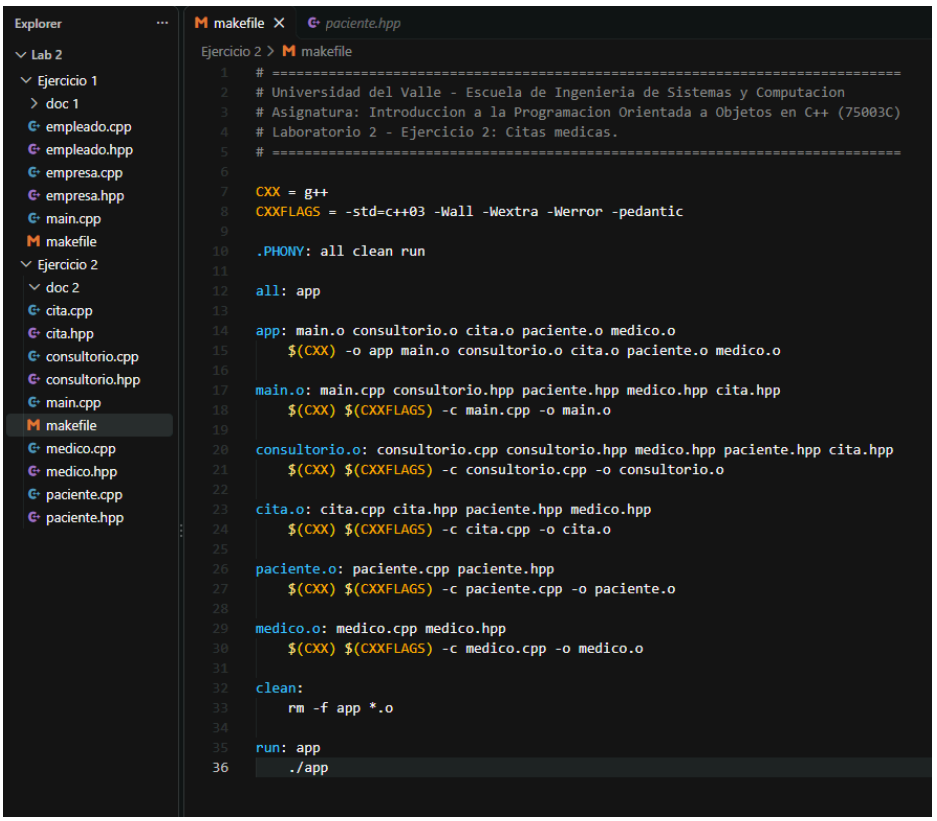
    int seleccion = 0;
    while (true)
    {
        std::cout << "Seleccione el numero de medico (1 - " << Consultorio::TOTAL_MEDIC
        if (std::cin >> seleccion)
        {
            if (seleccion >= 1 && seleccion <= Consultorio::TOTAL_MEDICOS)
            {
                std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
                break;
            }
            std::cout << "Error: Opcion fuera de rango.\n";
        }
        else
        {
            std::cout << "Error: Ingrese un numero valido.\n";
            std::cin.clear();
            std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
        }
    }

    int pos_medico = seleccion - 1;

    if (consultorio.AgendarCita(paciente, pos_medico, fecha, hora))
    {
        std::cout << "\n========================================\n";
        std::cout << "          !CITA CONFIRMADA EXITOSAMENTE!          \n";
        std::cout << "========================================\n";
        const std::vector<Cita>& citas = consultorio.GetCitas();
        std::cout << citas.back().ResumenReserva() << "\n";
        std::cout << "========================================\n";
    }
    else
    {
        std::cout << "\n========================================\n";
        std::cout << "Error: El medico seleccionado NO esta disponible en esa fecha y h
```

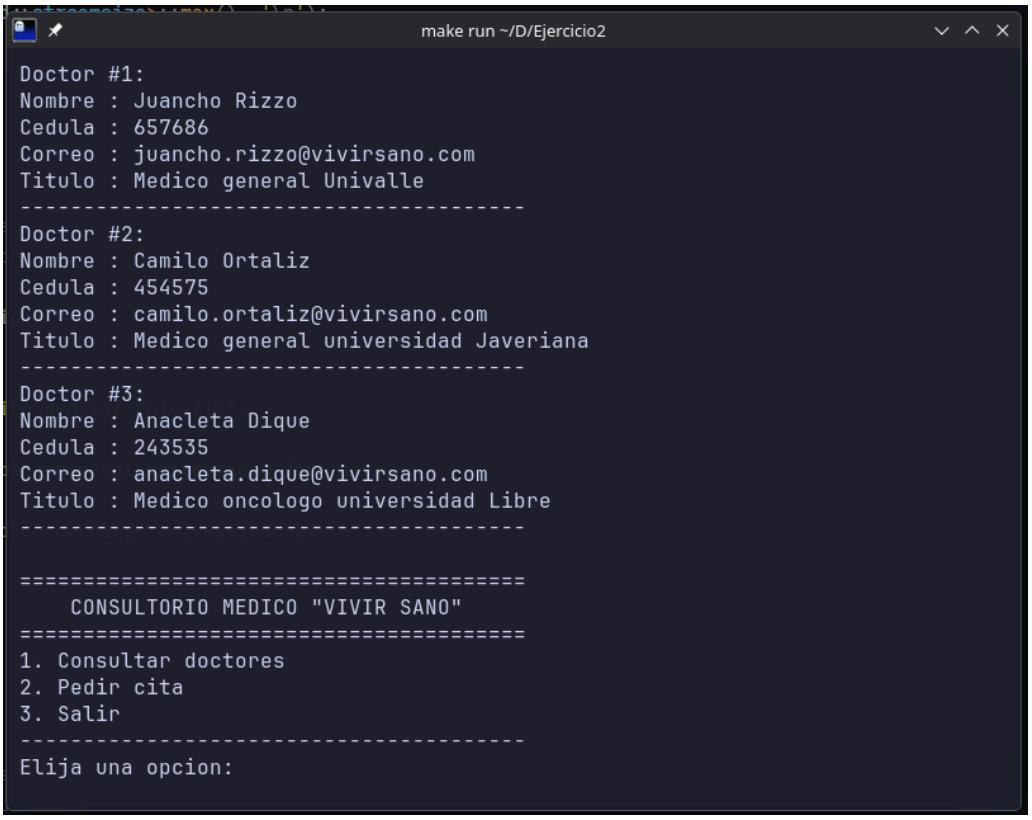

Explorer
Lab 2
Ejercicio 1
doc 1
empleado.cpp
empleado.hpp
empresa.cpp
empresa.hpp
main.cpp
makefile
Ejercicio 2
doc 2
cita.cpp
cita.hpp
consultorio.cpp
consultorio.hpp
main.cpp
makefile
medico.cpp
medico.hpp
paciente.cpp
paciente.hpp
Outline
Timeline
main.cpp
Ejercicio 2 > main.cpp
/**
* @brief Punto de entrada principal.
* @return Estado de finalizacion del programa.
*/
int main()
{
Consultorio consultorio("Vivir Sano");
CargarMedicosIniciales(consultorio);
int opcion = 0;
do
{
std::cout << "\n==\n";
std::cout << "   CONSULTORIO MEDICO \"VIVIR SANO\"   \n";
std::cout << "==\n";
std::cout << "1. Consultar doctores\n";
std::cout << "2. Pedir cita\n";
std::cout << "3. Salir\n";
std::cout << "--\n";
std::cout << "Elija una opcion: ";
if (std::cin >> opcion)
{
std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
switch (opcion)
{
case 1:
MostrarDoctores(consultorio);
break;
case 2:
PedirCita(consultorio);
break;
case 3:
std::cout << "\nGracias por usar el sistema del Consultorio Vivir Sano.
break;
default:
std::cout << "\nError: Opcion no valida. Por favor intente de nuevo.\n";
break;
}
}
else
{
std::cout << "\n==\n";
std::cout << "Error: El medico seleccionado NO esta disponible en esa fecha y h
std::cout << "==\n";
}
}
std::cout << "\nError: Ingrese un numero valido.\n";
std::cin.clear();
std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
} while (opcion != 3);
return 0;
}

makefile

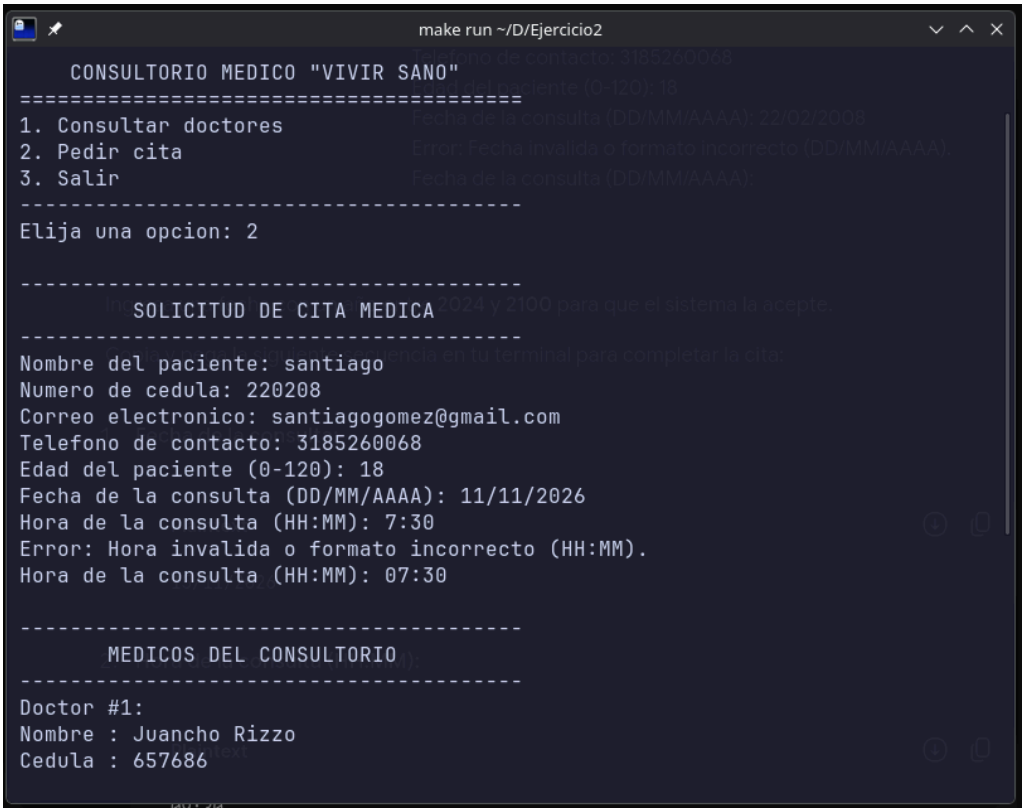

Explorer
Lab 2
Ejercicio 1
doc 1
empleado.cpp
empleado.hpp
empresa.cpp
empresa.hpp
main.cpp
makefile
Ejercicio 2
doc 2
cita.cpp
cita.hpp
consultorio.cpp
consultorio.hpp
main.cpp
makefile
medico.cpp
medico.hpp
paciente.cpp
paciente.hpp
makefile
paciente.hpp
Ejercicio 2 > makefile
==
Universidad del Valle - Escuela de Ingenieria de Sistemas y Computacion
Asignatura: Introduccion a la Programacion Orientada a Objetos en C++ (75003C)
Laboratorio 2 - Ejercicio 2: Citas medicas.
==
CXX = g++
CXXFLAGS = -std=c++03 -Wall -Wextra -Werror -pedantic
.PHONY: all clean run
all: app
app: main.o consultorio.o cita.o paciente.o medico.o
$(CXX) -o app main.o consultorio.o cita.o paciente.o medico.o
main.o: main.cpp consultorio.hpp paciente.hpp medico.hpp cita.hpp
$(CXX) $(CXXFLAGS) -c main.cpp -o main.o
consultorio.o: consultorio.cpp consultorio.hpp medico.hpp paciente.hpp cita.hpp
$(CXX) $(CXXFLAGS) -c consultorio.cpp -o consultorio.o
cita.o: cita.cpp cita.hpp paciente.hpp medico.hpp
$(CXX) $(CXXFLAGS) -c cita.cpp -o cita.o
paciente.o: paciente.cpp paciente.hpp
$(CXX) $(CXXFLAGS) -c paciente.cpp -o paciente.o
medico.o: medico.cpp medico.hpp
$(CXX) $(CXXFLAGS) -c medico.cpp -o medico.o
clean:
rm -f app *.o
run: app
./app

7. Evidencias

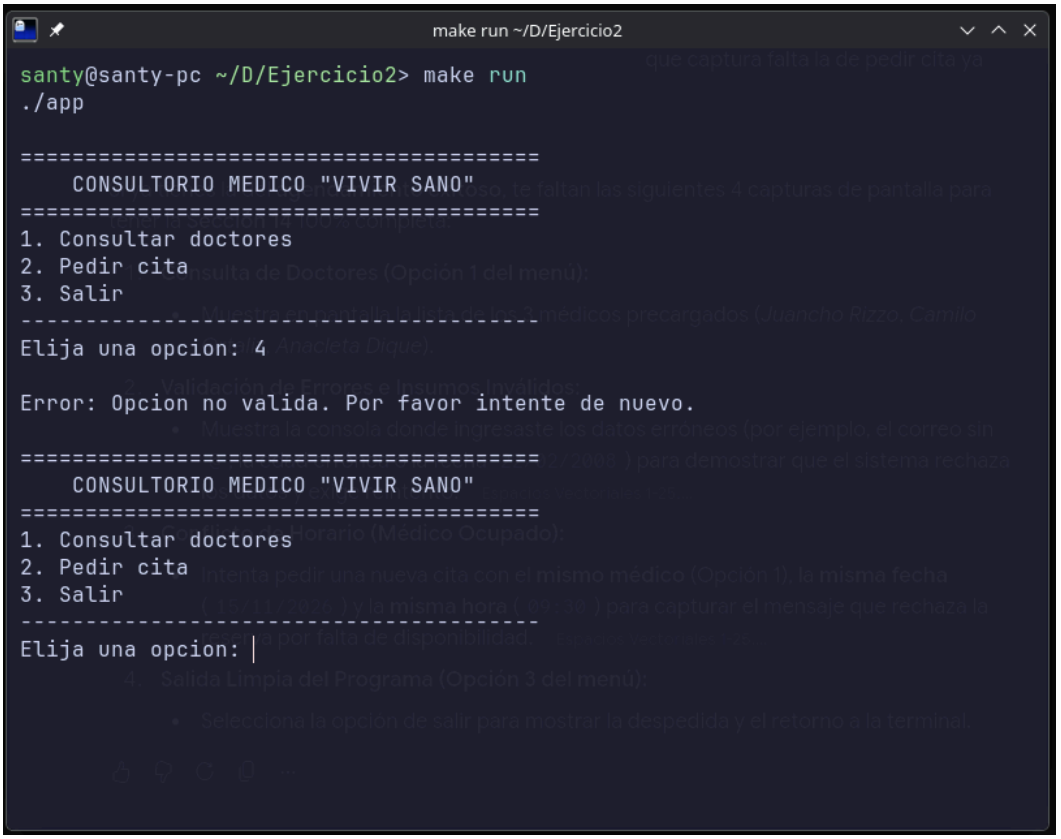
menu doctores



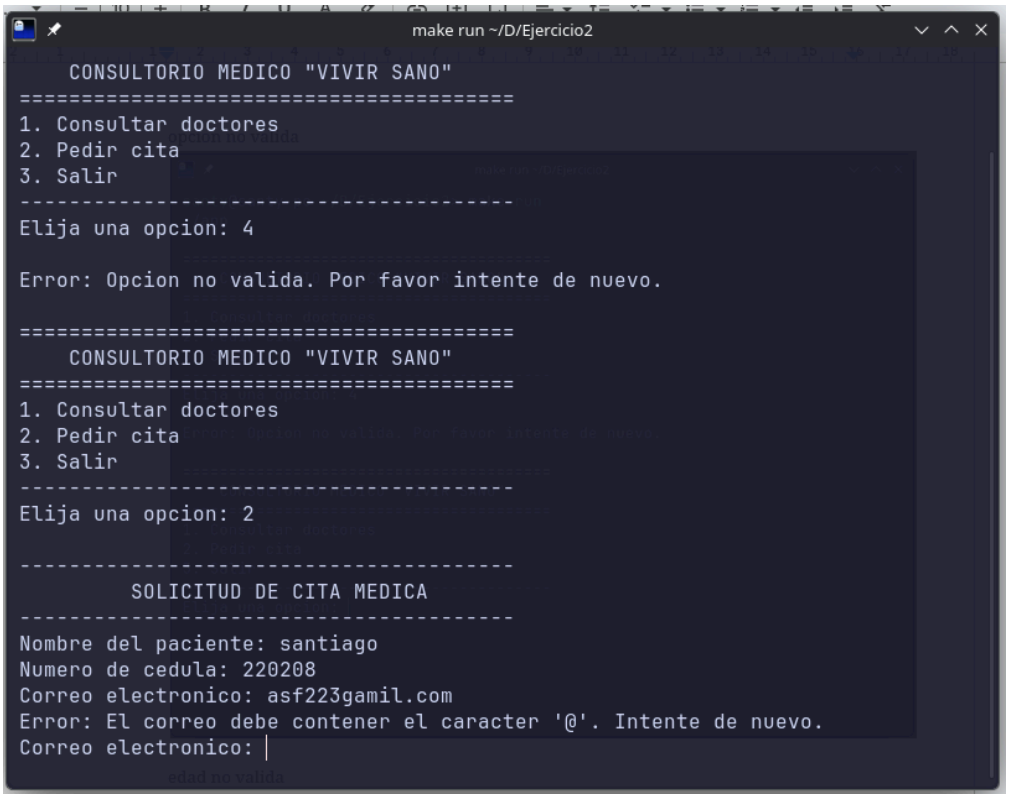
solicitar cita



opcion no valida



correo no valido



edad no valida

